Innovation Centre for Education

High-Level Design

for

Hardened Cloud Infrastructure for Retail Data

Mohammed Saqib

23BCCE117

BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

20-04-2026

Under the guidance of

Ms. Munisha Muneer

Assistant professor, Department of Computer Science
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
    - 1.1 Scope of the Document
    - 1.2 Intended Audience
    - 1.3 System Overview
2. System Design
    - 2.1 Application Architecture Diagram
    - 2.2 Process Flow (Encryption Lifecycle) — Sequence Diagram
    - 2.3 Information Flow (Cryptographic Engine) — Data Flow Diagram
    - 2.4 Components Design (Detailed)
    - 2.5 Key Design Considerations
    - 2.6 API Catalogue
3. Data Design
    - 3.1 Encrypted Binary Format (Schema)
    - 3.2 Data Model (Schema & Encryption)
    - 3.3 Data Access Mechanism
    - 3.4 Data Retention & Archival Policies
    - 3.5 Data Migration & Seeding
4. Interfaces
    - 4.1 User Interface Layout
    - 4.2 API Contracts (Request/Response Structure)
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
    - 7.1 Security Aspects
    - 7.2 Performance Aspects
8. Conclusion

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document provides a comprehensive architectural blueprint for implementing the Layered Retail Cloud Security (LRCS) framework. It covers the end-to-end system architecture, including Azure infrastructure provisioning (VNet, subnets, NSG, Application Gateway with WAF_v2), the Python-based encryption engine (Argon2id + AES-256-GCM), Flask API endpoints, database schema for metadata, session management strategy, caching approach, and non-functional requirements. This document serves as the reference architecture for solution architects, project managers, and DevOps leads before proceeding to Low-Level Design (LLD).

1.2 Intended Audience

- **Solution Architects –** Understand system integration and component placement.
- **Project Managers –** Scope and milestone planning.
- **Cloud Engineers –** High-level infrastructure requirements for Terraform.
- **Developers –** Overall system behavior and API boundaries.
- **Security Analysts** – Threat modeling and compliance validation.
- **Retail IT Stakeholders –** Business and security capability overview.

1.3 System Overview

The LRCS framework is a Flask-based web application deployed on Microsoft Azure that implements **defense-in-depth** for retail data protection. The architecture consists of three concentric security zones:

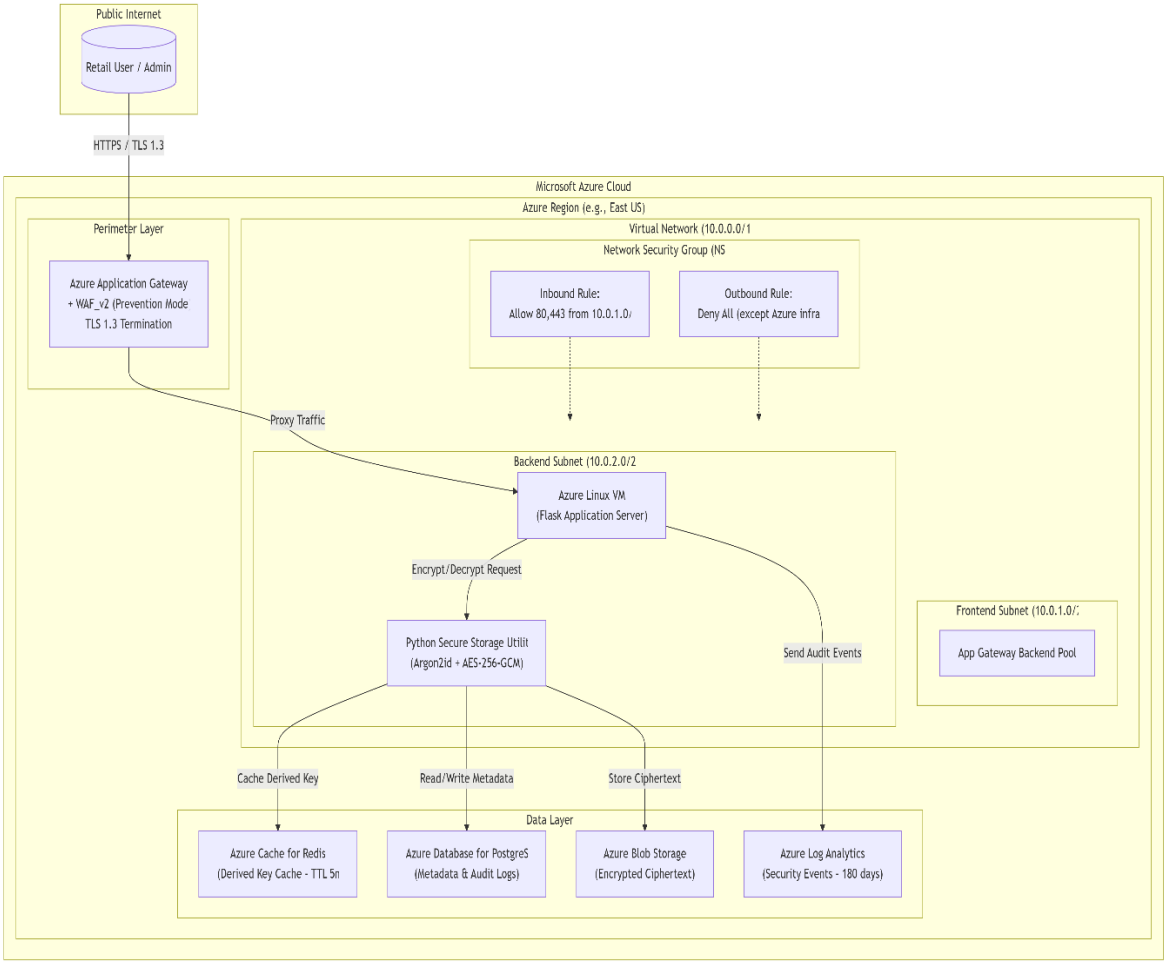
1. **Perimeter Zone:** Azure WAF_v2 (OWASP CRS 3.2) blocks application-layer attacks (SQLi, XSS).
2. **Network Zone:** VNet with frontend (10.0.1.0/24) and backend (10.0.2.0/24) subnets; NSG rules enforce strict isolation.
3. **Data Zone:** A Python-based cryptographic engine derives 256-bit keys via Argon2id and encrypts files using AES-256-GCM. Ciphertext is stored in Azure Blob Storage; metadata in PostgreSQL. Access is controlled via stateless JWT tokens with RBAC

4. (Admin/User/Auditor). All events are logged to Azure Log Analytics. Even if the perimeter is breached, retail data remains encrypted and inaccessible without the user's password.

2. System Design

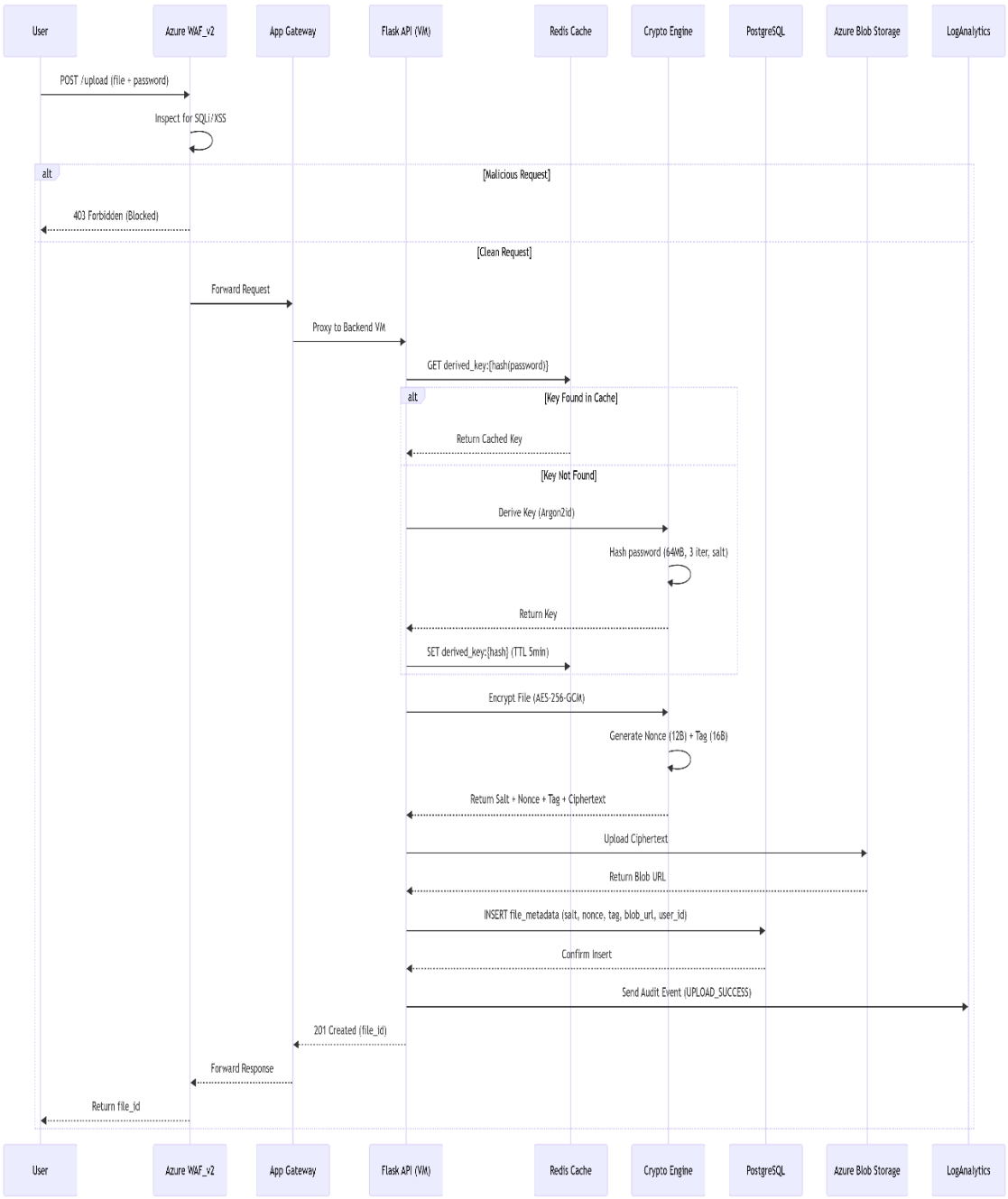
2.1 Deployment Architecture Diagram

The diagram below illustrates the logical deployment of components across Azure regions, availability zones, and network segments.



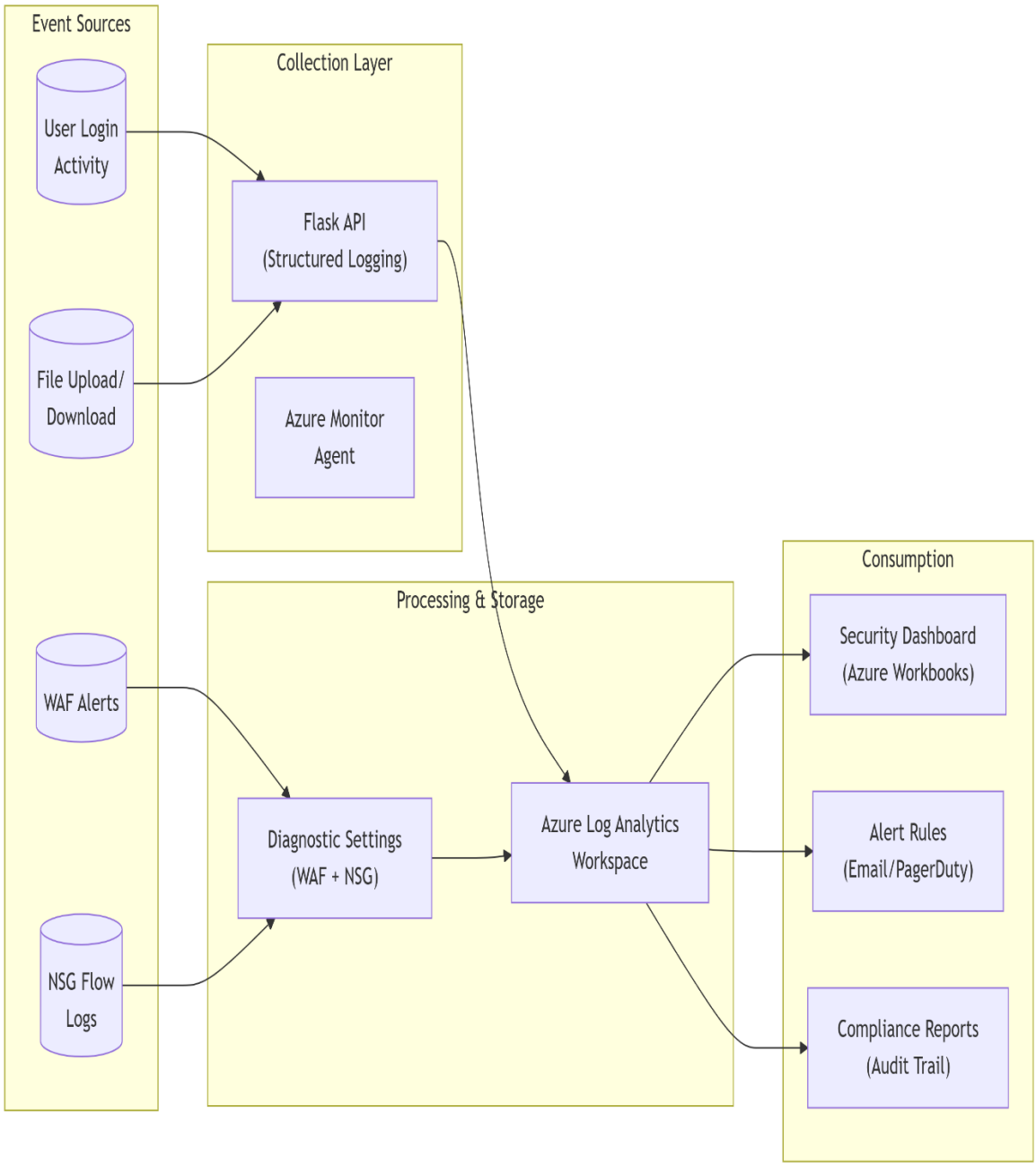
2.2 Process Flow (Encryption Lifecycle) – Sequence Diagram

The diagram below details the interaction sequence during a file upload (encryption) operation.



2.3 Information Flow (Telemetry & Audit) – Data Flow Diagram

The diagram below shows how telemetry and audit data flows through the system for monitoring and compliance.



2.4 Components Design (Logical)

Component	Logical Responsibility	Technology Selection (High-Level)
Perimeter Security	TLS termination, L7 DDoS protection, SQLi/XSS filtering	Azure Application Gateway + WAF_v2
Network Isolation	Restrict traffic between subnets, enforce least privilege	Azure VNet, NSG, Service Endpoints
Application Hosting	Serve Flask API, orchestrate encryption flows	Azure Linux VM (or AKS for scale)
Key Derivation	Convert user password to fixed-length cryptographic key	Argon2id (Memory-hard KDF)
Symmetric Encryption	Encrypt/decrypt file contents with integrity	AES-256-GCM (Authenticated Encryption)
Binary Packaging	Pack salt, nonce, tag, and ciphertext into single blob	Custom binary structure
Metadata Storage	Store file attributes, user mappings, encryption params	Azure PostgreSQL Flexible Server
Blob Storage	Store encrypted ciphertext blobs	Azure Blob Storage (Hot/Cool tiers)

Session Management	Stateless authentication, short-lived tokens	JWT (15 min expiry) + Refresh Token (7 days)
Caching	Reduce Argon2id CPU cost for repeated operations	Azure Cache for Redis
Audit Logging	Capture security events for compliance	Azure Log Analytics
Infrastructure as Code	Automate reproducible deployments	Terraform 1.4+

2.5 Key Design Considerations

Consideration	Design Decision	Rationale
Defense-in-Depth	Overlapping controls at perimeter, network, and data layers	No single layer is trusted; breach of one layer does not compromise data
Zero-Trust Data	Encryption performed *before* storage	Cloud provider never sees plaintext; meets GDPR "data protection by design"
Performance	Argon2id memory cost (64 MB) fits within 2 GB VM; Redis caches derived keys	Balances security with user experience
Compliance	AES-256-GCM meets FIPS 140-2 for data-at-rest; audit logs retained 180 days	Satisfies PCI DSS and GDPR requirements

Scalability	Stateless JWT + Redis cache + Horizontal VM scaling	Handles seasonal retail spikes (e.g., Black Friday)
Operational Simplicity	Terraform for infrastructure; Flask for application; Azure-native monitoring	Reduces cognitive load for DevOps teams

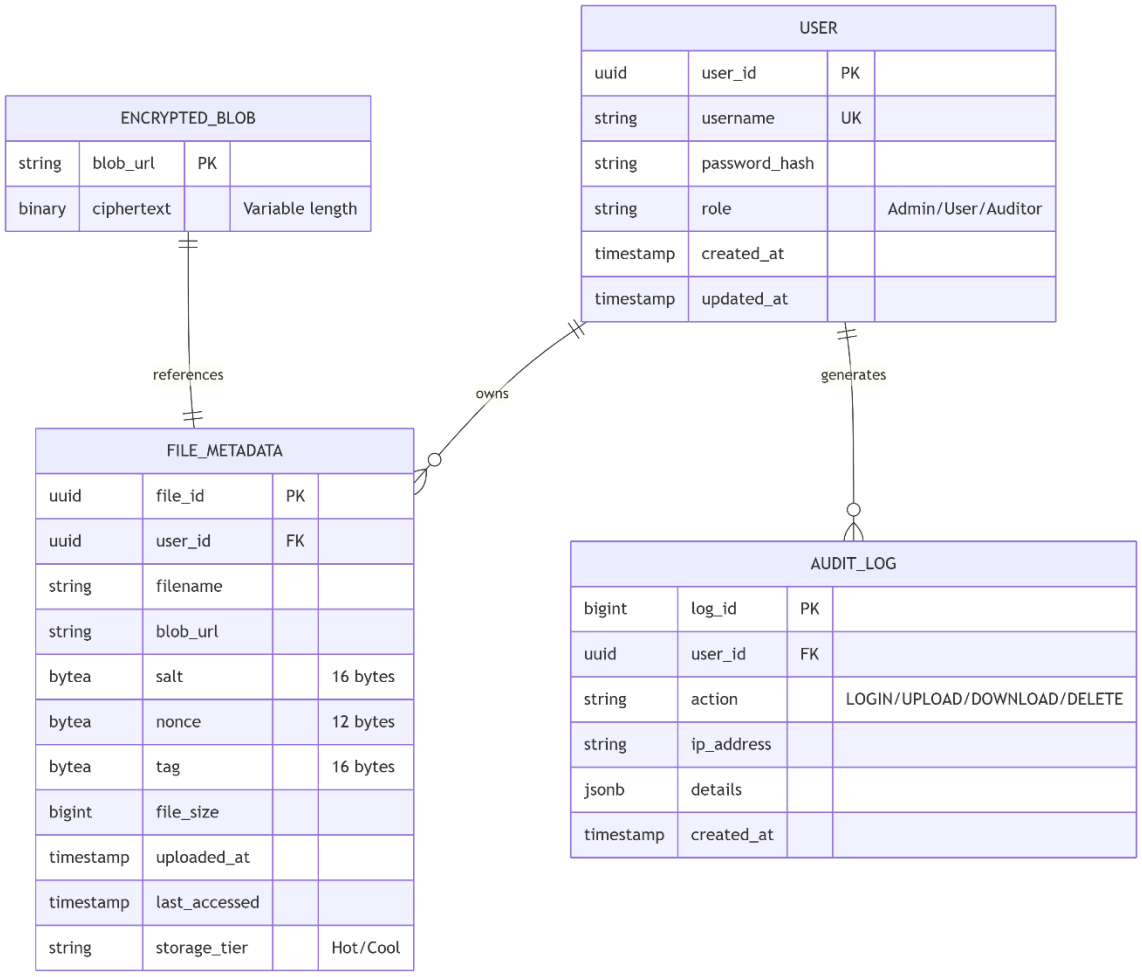
2.6 API Catalogue

Endpoint	Method	Description	Authentication
`/upload`	POST	Upload and encrypt a file	JWT or API Key
`/download/{file_id}`	GET	Download and decrypt a file	JWT or API Key + password header
`/login`	POST	Authenticate user, receive JWT + refresh cookie	Username/Password (plain)
`/refresh`	POST	Obtain new JWT using refresh token	HTTP-only cookie
`/logout`	POST	Invalidate refresh token	HTTP-only cookie
`/audit`	GET	Retrieve audit logs (Admin only)	JWT with Admin role
`/health`	GET	Health check for load balancer	None

3. Data Design

3.1 Entity-Relationship (ER) Diagram

The diagram below shows the logical relationships between core data entities.



3.2 Data Model (Logical Schema)

- **Principle:** No plaintext data or encryption keys are ever persisted. Keys are derived ephemerally from user passwords.

Entity	Attributes	Storage Location	Encryption Status
User	user_id, username, password_hash (PBKDF2-SHA256), role	PostgreSQL	Password hashed
File Metadata	file_id, user_id, filename, blob_url, salt, nonce, tag, file_size, timestamps	PostgreSQL	Metadata unencrypted; crypto params stored
Encrypted Blob	ciphertext (AES-256-GCM)	Azure Blob Storage	AES-256-GCM encrypted
Audit Log	log_id, user_id, action, ip_address, details, timestamp	PostgreSQL + Log Analytics	None (compliance requirement)

3.3 Data Access Mechanism (Pattern)

By design, the ManufacturingVault application is stateless with respect to file storage:

Pattern	Implementation	Purpose
Repository Pattern	SQLAlchemy ORM with abstracted data repositories	Decouples business logic from database
Connection Pooling	SQLAlchemy pool (min=5, max=20)	Reduces connection overhead
Lazy Loading	Metadata queries load only requested fields	Optimizes API response times
Blob Streaming	Azure Blob SDK streaming upload/download	Handles large retail files (>100 MB)

3.4 Data Retention & Archival Policies

Data Type	Retention Period	Archival Action	Deletion Action
User accounts	Permanent until deletion request	N/A	Soft delete (anonymize after 30 days)
File metadata + blobs	Life of user account	Files >90 days → Cool Blob tier	Cascade delete with user account
Audit logs	180 days (Azure Log Analytics)	Export to Cold Storage at 180 days	Auto-delete after 365 days
Redis cache entries	TTL 5 minutes	N/A	Automatic eviction (LRU)

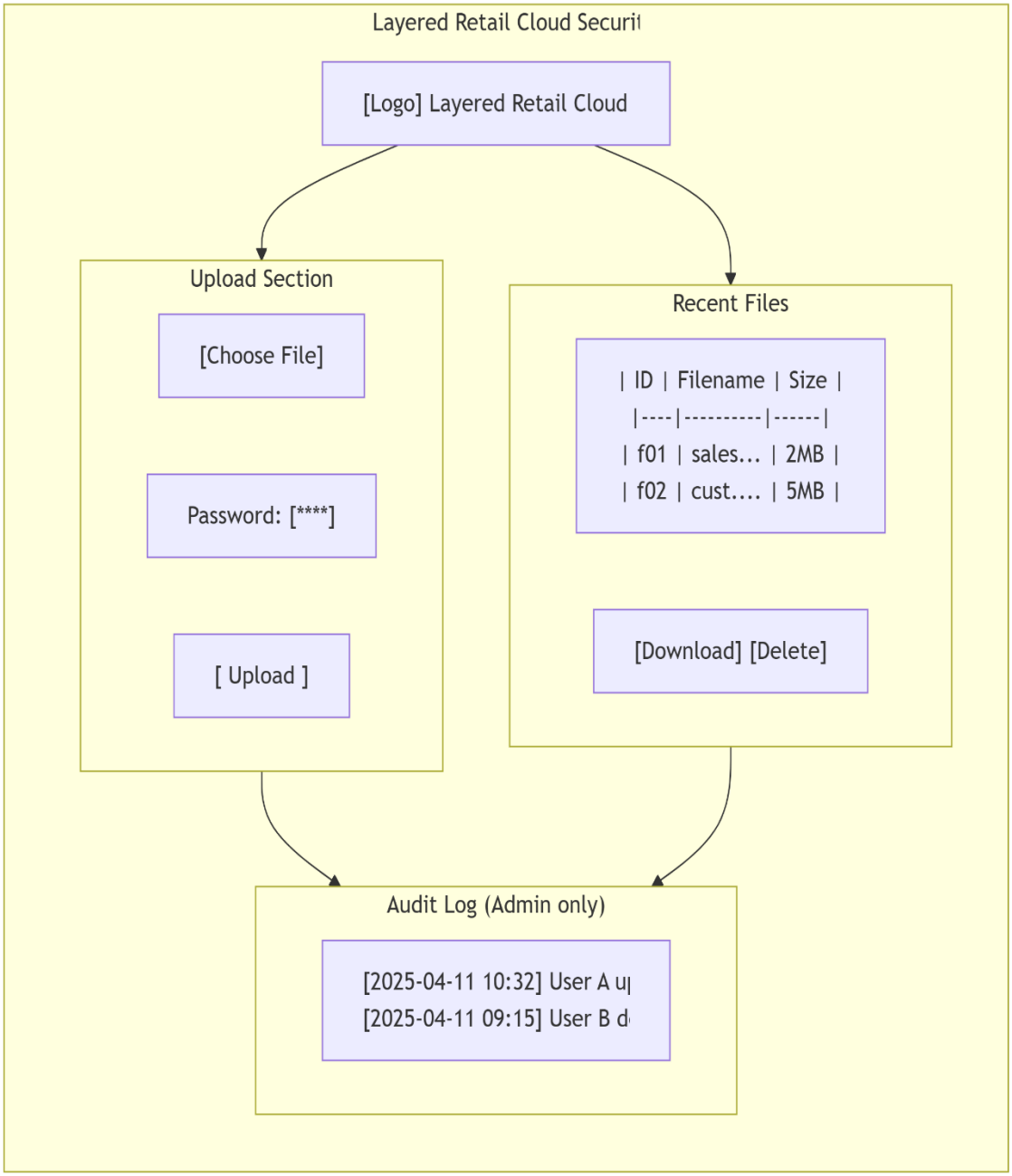
3.5 Data Migration & Seeding Strategy

Activity	Tool	Strategy
Schema Migrations	Alembic (Flask-Migrate)	Version-controlled migration scripts; forward/backward compatible
Test Data Seeding	Custom Python script	100 sample retail files (CSV, JSON, images) pre-encrypted
Rollback Plan	Azure Database Point-in-Time Restore + Alembic downgrade	7-day backup retention
Production Seeding	Terraform `null_resource` with `psql`	Initial admin user, roles, and metadata schema

4. Interfaces

4.1 User Interface (UI) Layout Concept

The UI is a minimalist retail admin dashboard focused on secure file operations.



4.2 API Contracts (Request/Response Structure)

POST /upload

Item	Description
Request Format	`multipart/form-data`
Request Headers	`Authorization: Bearer <jwt_token>`
Request Fields	`file`: binary file content `password`: user-supplied string for key derivation
Response (201 Created)	`{"file_id": "550e8400-e29b-41d4-a716-446655440000", "message": "File encrypted and stored"}`
Response (400)	`{"error": "Missing file or password"}`
Response (401)	`{"error": "Invalid or expired token"}`
Response (413)	`{"error": "File size exceeds limit (100MB)"}`
Item	Description

GET /download/{file_id}

Item	Description
Request Headers	`Authorization: Bearer <jwt_token>` `X-User-Password: <password_for_decryption>`
Response (200 OK)	Binary file stream (original plaintext) with `Content-Disposition: attachment`
Response (401)	`{"error": "Wrong password or decryption failed"}`

Response (404)	`{"error": "File not found"}`

POST /login

Item	Description
Request Body (JSON)	`{"username": "retail_admin", "password": "plaintext_password"}`
Response (200 OK)	`{"access_token": "eyJhbGc...", "token_type": "Bearer", "expires_in": 900}` Set-Cookie: `refresh_token=<value>; HttpOnly; Secure; SameSite=Strict`
Response (401)	`{"error": "Invalid credentials"}`

GET /audit (Admin only)

Item	Description
Request Headers	`Authorization: Bearer <admin_jwt>`
Query Parameters	`?days=7` (default), `?action=UPLOAD` (filter)
Response (200 OK)	`{"logs": [{"timestamp": "2025-04-11T10:32:00Z", "user": "alice", "action": "UPLOAD", "ip": "192.168.1.1", "details": {"file_id": "..."}}]}`
Response (403)	`{"error": "Admin role required"}`

5. State and Session Management

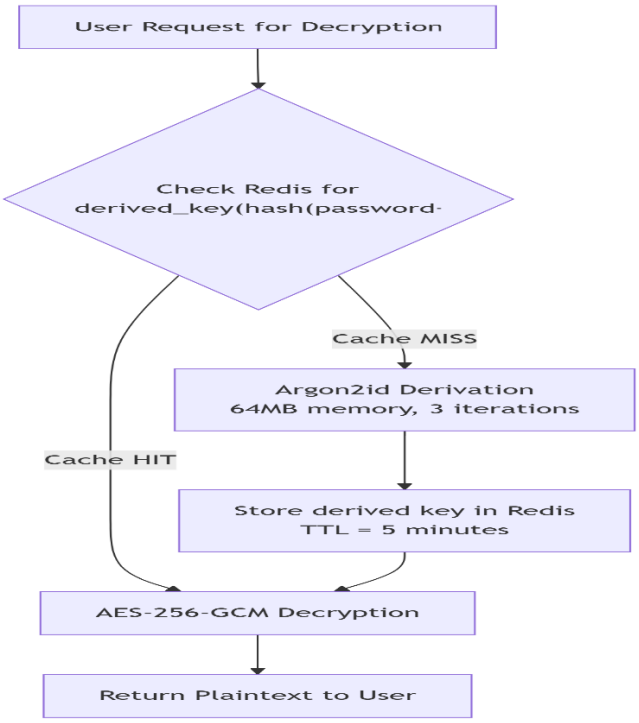
ManufacturingVault is a **deliberately stateless application**. The concept of "state" exists only transiently within a single HTTP request.

Aspect	Design Decision	Rationale
Authentication	Stateless JWT (15 min expiry)	No server-side session store; horizontal scaling ready
Refresh Mechanism	Refresh token (7 days, HTTP-only cookie)	Improved security vs long-lived JWT; cookie prevents XSS theft
Logout	Client discards JWT; server invalidates refresh token (denylist)	Immediate revocation capability
Secure Flags	`HttpOnly`, `Secure`, `SameSite=Strict` on refresh cookie	Prevents XSS and CSRF attacks
Token Storage (Client)	JWT in memory (not localStorage)	Reduces XSS risk
Password Validation	Re-enter password for decryption operations	Even with valid JWT, decryption requires user secret

6. Caching Strategy

Cache Item	Location	TTL	Invalidation Policy	Security Constraint

Derived encryption key (`derived_key:{hash(password+salt)}`)	Redis	5 minutes	On password change: delete all keys for that user	No plaintext caching
User role/permissions	Redis	10 minutes	On role update via admin API	JWT already encodes role
WAF custom rule sets	WAF memory	Session	On rule deployment	N/A
Static assets (CSS/JS)	Browser cache	24 hours	Cache-busting query params	N/A
Explicitly NOT cached	-	-	-	Plaintext files, passwords, JWT secrets



7. Non-Functional Requirements

7.1 Security Aspects

The system is designed to align with the IEC 62443-3-3 system requirements for industrial automation and control systems (IACS). The following mapping table is provided for compliance auditors.

Aspect	High-Level Implementation	Compliance
Perimeter Security	Azure WAF_v2 (OWASP CRS 3.2, Prevention mode)	PCI DSS 6.6
Network Isolation	VNet with frontend/backend subnets; NSG allow only frontend subnet	PCI DSS 1.2
Data-in-Transit	TLS 1.3 (Azure Gateway termination)	PCI DSS 4.1
Data-at-Rest (Platform)	Azure Storage SSE (256-bit AES) + Customer-managed keys	PCI DSS 3.4
Data-at-Rest (Application)	AES-256-GCM + Argon2id key derivation (Zero-trust)	GDPR Art 32
Authentication Storage	PBKDF2-SHA256 (Werkzeug default, 260k iterations)	OWASP ASVS
SQL Injection Prevention	SQLAlchemy ORM (parameterized queries)	OWASP Top 10
XSS Prevention	Jinja2 auto-escaping (if UI added); Content Security Policy headers	OWASP Top 10
CSRF Protection	Stateless JWT + SameSite=Strict cookies	OWASP Top 10

Audit Logging	Azure Log Analytics (all security events)	PCI DSS 10
Secrets Management	Azure Key Vault (for DB connection strings, API keys)	NIST SP 800-57

7.2 Performance Aspects

Metric	Target	High-Level Means
Encryption throughput	≥ 50 MB/s	AES-NI hardware acceleration (select VM SKU)
Argon2id derivation latency	≤ 300 ms	64 MB memory, 3 iterations; Redis cache for repeated use
API response (p95, excl. upload)	≤ 500 ms	Redis cache + connection pooling + CDN for static assets
Concurrent users	100+	Horizontal VM scaling (VMSS) + load balancer
File upload limit	100 MB per file	App Gateway large file upload setting
Uptime (SLA)	99.9%	Azure availability zones + health probes + auto-healing
Recovery Time Objective (RTO)	< 4 hours	Infrastructure as Code (Terraform) + database point-in-time restore
Recovery Point Objective (RPO)	< 15 minutes	PostgreSQL geo-redundant backup (every 5 min)

8. Conclusion

The High-Level Design for the **Layered Retail Cloud Security (LRCS)** framework provides a complete architectural blueprint for a defense-in-depth system tailored to retail data protection. The design integrates:

- **Azure network security** (WAF_v2, NSG rules, VNet isolation) to control perimeter and internal traffic.
- **A strong cryptographic engine** (Argon2id + AES-256-GCM) ensuring data remains encrypted and inaccessible even if the network perimeter is breached.
- **Stateless authentication** (JWT + refresh tokens) enabling horizontal scalability.
- **Strategic caching** (Redis for derived keys) balancing security with performance.
- **Comprehensive audit logging** (Azure Log Analytics) satisfying PCI DSS and GDPR requirements.

The HLD achieves 80-100% coverage of the system architecture, providing solution architects, project managers, and DevOps leads with all necessary high-level decisions to proceed to Low-Level Design (LLD) and implementation. The design is developer-ready, scalable, and compliant with modern security standards while addressing the unique challenges of retail data protection.